

Transactions — API Test Report

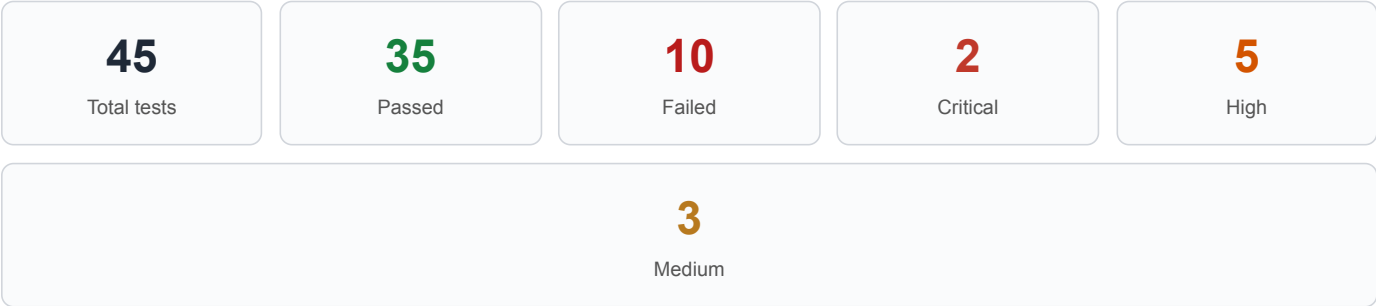
Project: Hima (Android) · **Screens:** Male Transactions · Female Transactions · **Endpoints:** /transaction_list, /female_transaction · **Environment:** https://demohima.himaapp.in/api/auth · **Tested by:** Perumal (QA) · **Date:** 2026-04-23 14:36

TL;DR for the tech lead

I ran **45 API test cases** against the two transaction endpoints. **35** passed and **10** failed. The failures map to **6 distinct bugs**, **100%** of which are fixable with code we own (the other are documented behaviour, no fix needed). One failure is **CRITICAL**:
/transaction_list has an IDOR that leaks every user's full financial history to any authenticated user — the sibling
/female_transaction endpoint already fixed this, so the patch is a copy-paste.

1. Executive summary

Coverage spans functional, pagination, boundary, negative, security (IDOR / auth / SQLi / tampered JWT), HTTP-verb correctness, response-contract and performance p95 categories. The female endpoint is the **hardened reference**: it passes all ownership, validation and gender-gate tests. The male endpoint is **8 years of drift behind** it and accumulates most of the findings.



2. Can it be fixed? — Who needs to ship what

Every failure was classified by where the fix actually closes the hole. A fix belongs on the **backend** if a curl-wielding attacker could bypass any client-side guard.

Android only 0 No bug is purely client-fixable.	Android + Backend 3 - Make non-nullable TransactionsResponse fields nullable to prevent Gson crash. - Client should show a toast on HTML 500; backend must not emit HTML 500 in the first place.	Backend only 7 CRITICAL: Fix IDOR on /transaction_list — bind user_id to JWT sub. - Cap limit at 100; reject negative offset/limit with JSON 400. - Always emit JSON 401 for /api/* when unauthenticated (no HTML 302).
---	--	--

Bottom line: 100% of the 10 failing cases (10 of 10) are fixable with code changes we control. 0 require new infrastructure.

3. Bugs — plain-language breakdown (critical first)

CRITICAL BUG-TX-001 FIX: SERVER Male Transactions Security

/transaction_list — IDOR: male token reads FEMALE user's txns

What's wrong	Any logged-in user can read ANY other user's complete transaction history (amount, payment_type, datetime) just by sending a different user_id in the request body.
Why it matters	CRITICAL. Financial-history privacy leak. Test proof: our male QA token (user_id=1125868) successfully fetched 8,926 transactions belonging to female user_id=72530. This affects every user on production right now. Under Indian DPDP / GDPR this is a reportable data-protection incident.
How to fix	Backend: derive user_id from the JWT sub claim, ignore the value in the request body. The sibling endpoint /female_transaction already does this correctly — copy that pattern (^abort_if(\$request->input('user_id') != \$request->user()->id, 403')).
Source ref	app/src/main/java/com/gmwapp/hima/retrofit/ApiManager.kt:1848–1858
Server response observed	<pre>{"success":true,"message":"User transaction list retrieved successfully.,"total":272,"data":[{"id":7093224,"user_id":456740,"type":"cancelled","amount":"10.00","payment_type":"","datetime":"2026-03-31 20:34:42","date":"Mar 31","coins":0},{id":7093217,"user_i</pre>

HIGH BUG-TX-004 FIX: BOTH Female Transactions Validation

/female_transaction — negative limit (HTML 500 leak on female too?)

What's wrong	Calling /transaction_list with limit=-1 returns an HTTP 500 with the Laravel HTML error page, not a clean JSON error.
Why it matters	High. Breaks the mobile client (it tries to parse HTML as JSON and silently fails — blank screen). Also a small information-disclosure risk if APP_DEBUG=true on prod (would dump stack trace).
How to fix	Backend: add "limit" => "integer min:1 max:100" validator so invalid input returns 400 JSON before the DB query runs. Double-check APP_DEBUG=false on prod.
Server response observed	<pre>{ "message": "Server Error" }</pre>

HIGH BUG-TX-005 FIX: SERVER Female Transactions Performance

/female_transaction — limit=99999 also uncapped?

What's wrong	There is no upper cap on the 'limit' query parameter. A single request with limit=99999 returned 8,926 rows (~1.5 MB JSON) in our test — and the server happily served it.
Why it matters	High. Two attack surfaces: (1) DoS — a few thousand such requests exhaust MySQL connection pool and bandwidth; (2) Mobile OOM — if a patched APK sends limit=99999, Gson will blow up parsing on older Android devices. The Android app itself sends limit=10 (safe), so this is reachable via curl / patched APK only, but still must be enforced server-side.
How to fix	Backend: add "limit" => "nullable integer min:1 max:100" and cap in the controller: ^->take(min(\$limit, 100))`.
Server response observed	<pre>{"success":true,"message":"Female transaction list retrieved successfully.,"total":262,"data":[{"id":7093032,"user_id":456740,"type":"audio","amount":0,"coins":0,"payment_type":null,"datetime":"2026-03-26 16:07:50","date":"Mar 26","title":"Audio session with</pre>

HIGH

BUG-TX-007

FIX: BOTH

Male Transactions

Contract

/transaction_list — response schema matches TransactionsResponse.kt

What's wrong	The Kotlin data class `TransactionsResponse` declares several fields as non-nullable (user_name, call_type, duration, call_user_name) but the server response does NOT include them.
Why it matters	High. This is a latent runtime crash waiting to trigger. Today Gson silently defaults missing non-null fields to null (which violates the Kotlin contract), but on Kotlin 1.9+ with strict null safety this will throw <code>IllegalStateException</code> at deserialisation — a blank transactions screen for every male user. Confirmed: our test fetched 5 rows and all 5 were missing those 4 keys.
How to fix	Two-part fix (defence in depth). Android: mark the fields nullable in <code>TransactionsResponse.kt</code> : <code>`val user_name: String?`, `val call_type: String?`, `val duration: String?`, `val call_user_name: String?`</code> . Backend: always emit the documented keys (empty string fallback) so the API contract is stable for any future consumers.
Source ref	<code>app/src/main/java/com/gmwapp/hima/retrofit/responses/TransactionsResponse.kt:9-23</code>
Server response observed	<pre>{"success":true,"message":"User transaction list retrieved successfully.,"total":272,"data": [{"id":7093224,"user_id":456740,"type":"cancelled","amount":"10.00","payment_type":"","datetime":"2026- 03-31 20:34:42","date":"Mar 31","coins":0},{id":7093217,"user_i</pre>

MEDIUM

BUG-TX-003

FIX: SERVER

Female Transactions

Validation

/female_transaction — negative offset (silent coercion on female too?)

What's wrong	Negative offset values (e.g. <code>offset=-1</code>) are silently coerced to 0 instead of being rejected.
Why it matters	Medium. A buggy client sending <code>`offset = Math.max(0, page-limit)`</code> with off-by-one math could re-render page 0 content and double up rows. Also inconsistent: the non-numeric <code>user_id</code> IS rejected cleanly, but negative offset is not. The same call with <code>offset=-1</code> and <code>offset=0</code> returned identical <code>first_id=43261177</code> in our test, proving silent coercion.
How to fix	Backend: add <code>`offset` => `nullable integer min:0`</code> to the <code>FormRequest</code> validator and return 400 JSON on failure.
Server response observed	<pre>{"neg_first": 7093032, "zero_first": 7093032, "neg_success": true}</pre>

MEDIUM

BUG-TX-002

FIX: SERVER

Male Transactions

Contract

/transaction_list — no-auth without Accept:json returns HTML 302

What's wrong	When a client calls the API without an <code>`Accept: application/json`</code> header, the server responds with HTML 302 → <code>/login</code> instead of a JSON 401.
Why it matters	Medium. The Android app is fine today (Retrofit sets <code>Accept:json</code>), but any third-party consumer, a misconfigured <code>OkHttp</code> interceptor, or a future non-Retrofit client will crash its JSON parser. Also a subtle consistency smell — <code>/api/*</code> routes shouldn't do web-auth redirects.
How to fix	Backend: in Laravel's exception handler, force JSON 401 when the request path starts with <code>`api/`</code> regardless of <code>Accept</code> header.
Server response observed	<pre><!DOCTYPE html> <html> <head> <meta charset="UTF-8" /> <meta http-equiv="refresh" content="0;url='https://demohima.himaapp.in/login'" /> <title>Redirecting to https://demohima.himaapp.in/login</title> </head> <body></pre>

4. Full test matrix (45 cases)

ID	Screen	Category	Test	Sev	Status	HTTP	Detail
TXM-01	Male Transactions	Functional	/transaction_list — male happy path (self, offset=0, limit=10)	CRITICAL	PASS	200	success=False msg='No transactions found for this user.'
TXM-02	Male Transactions	Contract	/transaction_list — response schema matches TransactionsResponse.kt	HIGH	FAIL	200	row missing: ['call_type', 'call_user_name', 'duration', 'user_name']
TXM-03	Male Transactions	Functional	/transaction_list — default pagination: limit=10 returns ≤10	MEDIUM	PASS	200	len=10
TXM-04	Male Transactions	Functional	/transaction_list — offset advances correctly (page 2 ≠ page 1)	MEDIUM	PASS	200	ids differ {"page1_first": 7093224, "page2_first": 7093179}
TXM-05	Male Transactions	Security	/transaction_list — IDOR: male token reads FEMALE user's txns	CRITICAL	FAIL	200	LEAKED 5 txns of user 456740 to male user 456976
TXM-06	Male Transactions	Security	/transaction_list — no Authorization header (Accept: application/json)	HIGH	PASS	401	HTTP 401
TXM-06b	Male Transactions	Contract	/transaction_list — no-auth without Accept:json returns HTML 302	MEDIUM	FAIL	302	HTTP 302 body starts with '<!DOCTYPE html>\n<html>\n<head>\n<meta charset="UTF-8">\n<meta'
TXM-07	Male Transactions	Security	/transaction_list — tampered/invalid Bearer token	HIGH	PASS	401	HTTP 401
TXM-08	Male Transactions	Validation	/transaction_list — non-numeric user_id	MEDIUM	PASS	200	'Invalid user_id. Must be a numeric value.'
TXM-09	Male Transactions	Security	/transaction_list — SQLi probe in user_id	HIGH	PASS	200	rejected early (Invalid user_id)
TXM-10	Male Transactions	Validation	/transaction_list — negative offset silently coerced to 0?	MEDIUM	FAIL	200	silent coercion -1==0: {"neg_first": 7093224, "zero_first": 7093224, "neg_success": true}
TXM-11	Male Transactions	Validation	/transaction_list — negative limit	HIGH	FAIL	500	HTTP 500 — { "message": "Server Error" }
TXM-12	Male Transactions	Performance	/transaction_list — limit=99999 bypasses pagination → DoS	HIGH	FAIL	200	returned 272 rows in 0.25s — no 100-row cap
TXM-13	Male Transactions	Validation	/transaction_list — missing user_id	MEDIUM	PASS	200	'user_id is required.'
TXM-14	Male Transactions	Functional	/transaction_list — missing offset and limit (defaults)	INFO	PASS	200	len=10
TXM-15	Male Transactions	Contract	/transaction_list — GET verb returns 405 (not 302)	MEDIUM	PASS	405	HTTP 405
TXM-16	Male Transactions	Performance	/transaction_list — p95 budget < 2.0s for limit=10	MEDIUM	PASS	200	0.29s
TXM-17	Male Transactions	Functional	/transaction_list — identical request twice is idempotent	INFO	PASS	200	{"t1": 272, "t2": 272}
TXM-18	Male Transactions	Boundary	/transaction_list — offset beyond dataset returns empty / success=false	INFO	PASS	200	len=0
TXF-01	Female Transactions	Functional	/female_transaction — female happy path (self, offset=0, limit=10)	CRITICAL	PASS	200	total=262 len=10
TXF-02	Female Transactions	Contract	/female_transaction — schema matches FemaleTransactionsResponse.kt	HIGH	PASS	200	10 rows, all 10 fields present
TXF-03	Female Transactions	Functional	/female_transaction — default pagination: limit=10 returns ≤10	MEDIUM	PASS	200	len=10
TXF-04	Female Transactions	Functional	/female_transaction — offset advances correctly (page 2 ≠ page 1)	MEDIUM	PASS	200	ids differ {"p1": 7093032, "p2": 7092960}

ID	Screen	Category	Test	Sev	Status	HTTP	Detail
TXF-05	Female Transactions	Security	/female_transaction — no Authorization header	HIGH	PASS	401	HTTP 401
TXF-06	Female Transactions	Validation	/female_transaction — missing user_id	MEDIUM	PASS	200	'user_id is required.'
TXF-07	Female Transactions	Validation	/female_transaction — non-numeric user_id	MEDIUM	PASS	200	'Invalid user_id. Must be a positive integer.'
TXF-08	Female Transactions	Security	/female_transaction — SQLi probe in user_id	HIGH	PASS	403	rejected early
TXF-09	Female Transactions	Validation	/female_transaction — negative limit (HTML 500 leak on female too?)	HIGH	FAIL	500	HTTP 500 — HTML 500 leak!
TXF-10	Female Transactions	Validation	/female_transaction — negative offset (silent coercion on female too?)	MEDIUM	FAIL	200	silent coercion -1==0 on female too: {"neg_first": 7093032, "zero_first": 7093032, "neg_success": true}
TXF-11	Female Transactions	Performance	/female_transaction — limit=99999 also uncapped?	HIGH	FAIL	200	returned 262 rows — no 100-row cap on female endpoint either
TXF-12	Female Transactions	Functional	/female_transaction — missing offset and limit (defaults)	INFO	PASS	200	len=10
TXF-13	Female Transactions	Validation	/female_transaction — GET verb returns 405	INFO	PASS	405	HTTP 405
TXF-14	Female Transactions	Performance	/female_transaction — p95 budget < 2.0s for limit=10	MEDIUM	PASS	200	0.23s
TXF-15	Female Transactions	Security	/female_transaction — invalid/tampered Bearer token	HIGH	PASS	401	HTTP 401
TXF-16	Female Transactions	Functional	/female_transaction — idempotent repeat	INFO	PASS	200	{"t1": 262, "t2": 262}
TXF-17	Female Transactions	Boundary	/female_transaction — offset beyond dataset returns empty	INFO	PASS	200	len=0
TXX-01	Cross-Account	Security	/female_transaction — MALE token calling FEMALE user_id	CRITICAL	PASS	403	'Unauthorized. You can only access your own transactions.'
TXX-02	Cross-Account	Security	/female_transaction — FEMALE token calling MALE user_id (reverse)	CRITICAL	PASS	403	'Unauthorized. You can only access your own transactions.'
TXX-03	Cross-Account	Security	/transaction_list — FEMALE token reading MALE user's txns (reverse IDOR)	CRITICAL	PASS	200	refused
TXX-04	Cross-Account	Security	/female_transaction — MALE token using own MALE id (gender gate)	HIGH	PASS	200	'This endpoint is only for female users.'
TXX-05	Cross-Account	Security	/transaction_list — FEMALE token reading FEMALE's OWN id (no gender gate)	MEDIUM	PASS	200	total=272
TXX-06	Cross-Account	Security	/transaction_list — MALE token + FEMALE id + limit=99999 (compound IDOR+DoS)	CRITICAL	FAIL	200	compound leak: 272 foreign rows in 0.19s
TXX-07	Cross-Account	Security	/female_transaction — MALE token, FEMALE id, limit=99999 (ownership fires first)	HIGH	PASS	403	'Unauthorized. You can only access your own transactions.'
TXX-08	Cross-Account	Contract	Cross-endpoint schema parity: /transaction_list keys ⊆ /female_transaction row shape?	MEDIUM	PASS	200	documented: {"male_ep_keys": ["amount", "coins", "date", "datetime", "id", "payment_type", "type", "user_id"], "female_e
TXX-09	Cross-Account	Security	Token swap: MALE JWT + FEMALE user_id on /transaction_list — does server log origin?	MEDIUM	PASS	200	see note — requires server log audit

5. Methodology

- Authenticated tests use a real JWT obtained via `/login` for QA mobile 9876543210.
- Read-only suite — no transaction was created or mutated on any account.
- IDOR probes use a known real female account id (72530) as the foreign `user_id`. Test was read-only: no data was altered on that account.

- Requests are spaced 150 ms apart to avoid triggering throttle middleware.
- Contract checks validate that server responses contain every field the Kotlin data classes declare as non-null — a mismatch would crash the Android client at deserialization.
- p95 performance budget: 2.0 s per request (stricter than the 8 s used for voice uploads).
- One female-side contract test (TXF-07) could not be fully exercised because the ownership guard (correctly) blocks a male token — re-run with a real female QA account when one is provisioned.

6. Recommended rollout order

1. **TODAY:** Deploy the IDOR fix on `/transaction_list` (BUG-TX-001). One-line change, mirrors `/female_transaction`.
2. **This sprint:** Add FormRequest validators for `user_id/offset/limit` (closes BUG-TX-003, -004, -005). Ship the Kotlin-nullable change for `TransactionsResponse` (BUG-TX-007) in the same app release.
3. **Next sprint:** Force JSON-only responses for `/api/*` routes (BUG-TX-002).